R.M.K
GROUP OF ENGINEERING INSTITUTIONS



R.M.K
GROUP OF INSTITUTIONS







R.M.K
GROUP OF
INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22CS702
DATA ANALYTICS

Department: CSE/IT

Batch/Year: 2023-2027(CSE) & 2024-28(IT) / IV Year(CSE) & III Year (IT)

Created by:

1. Dr.T.Sethukarasi – Professor & Head – RMKEC (CSE)
2. M.Thiyagarajan – Assistant Professor – RMKEC (CSE)
3. Ms.B.Bharathi – Assistant Professor – RMKEC (CSE)
4. Ms.K.Selvi – Assistant Professor – RMKEC (IT)
5. Dr.A.Gnanasekar – Associate Professor – RMDEC (CSE)
6. Ms.D.M.Kalai Selvi – Assistant Professor – RMDEC (CSE)
7. Ms. N. Arockia Rosy – Assistant Professor – RMDEC (IT)
8. Dr.M.Arun Manickaraja–Associate Professor – RMKCET (CSE)
9. Dr.Pranamita Nanda – Associate Professor – RMKCET (CSE)

R.M.K GROUP OF INSTITUTIONS

1.Table of Contents

S.NO	Topic	Page No.
1.	Contents	5
2.	Course Objectives	6
3.	Pre-Requisites	7
4.	Syllabus	8
5.	Course outcomes	9
6.	CO- PO/PSO Mapping	11
7.	Lecture Plan	13
8.	Activity based learning	14
9.	Lecture notes	16
10.	Video Links	50
11.	Assignments	51
12.	Part A Q & A	53
13.	Part B Qs	60
14.	Supportive online Certification courses	61
15.	Real time Applications in day to day life and to Industry	62
16.	Contents beyond the Syllabus	64
17.	Assessment Schedule	67
18.	Prescribed Text Books and Reference Books	68
19.	Mini Project Suggestions	69
20.	GATE Questions	70

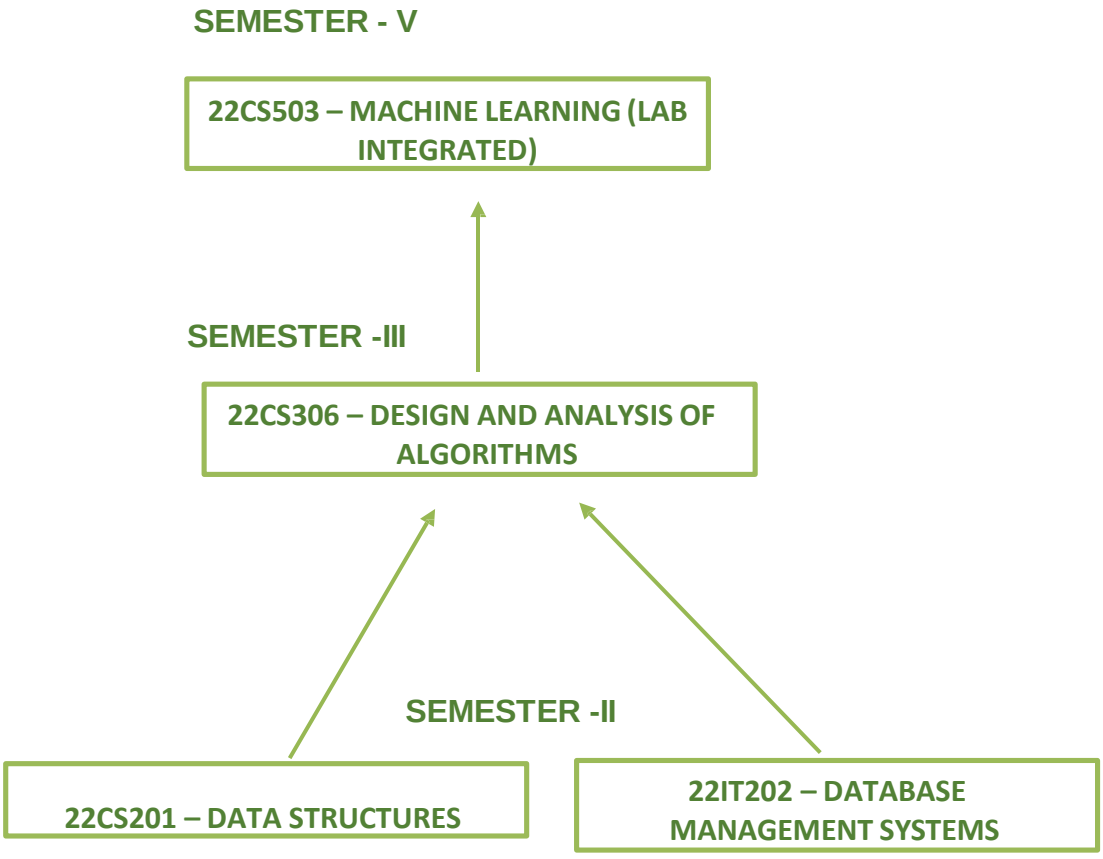
R.M.K GROUP OF INSTITUTIONS

2.Course Objectives

- To explain the fundamentals of big data and data analytics
- To discuss the Hadoop framework
- To explain about exploratory data analysis and data manipulation tools and use it for developing applications
- To analyze and interpret streaming data
- To discuss various applications of data analytics

R.M.K GROUP OF INSTITUTIONS

3.PRE-REQUISITES



4.SYLLABUS

22CS702 **DATA ANALYTICS** **L T P C**

3 0 2 4

UNIT I INTRODUCTION 9 + 6

Evolution of Big Data-Definition of Big Data-Challenges with Big Data- Traditional Business Intelligence (BI) versus Big Data-Introduction to big data analytics-Classification of Analytics-Analytics Tools- Importance of big data analytics.

UNIT II HADOOP FRAMEWORK 9+ 6

Introducing Hadoop- RDBMS versus Hadoop- Hadoop Overview-HDFS (Hadoop Distributed File System)- Processing Data with Hadoop- Managing Resources and Applications with Hadoop YARN - Interacting with Hadoop Ecosystem.

UNIT III EXPLORATORY DATA ANALYSIS 9+6

EDA fundamentals – Understanding data science – Significance of EDA – Making sense of data – Comparing EDA with classical and Bayesian analysis – Software tools for EDA –Data transformation techniques - Introduction to NoSQL – MongoDB: RDBMS Vs MongoDB – Data Types – Query Language – Hive – Hive Architecture – Data Types – File Formats - Hive Query Language (HQL) – RC File Implementation – User Defined Functions.

UNIT IV MINING DATA STREAMS 9+6

The data stream model – stream queries-sampling data in a stream-general streaming problem- filtering streams-analysis of filtering- dealing with infinite streams- Counting Distance Elements in a Stream – Estimating Moments – Counting Ones in Window – Decaying Windows.

UNIT V APPLICATIONS 9+6

Application: Sales and Marketing – Industry Specific Data Mining – microRNA Data Analysis Case Study – Credit Scoring Case Study – Data Mining Nontabular Data.

TOTAL: 75 PERIODS

R.M.K GROUP OF INSTITUTIONS

5.COURSE OUTCOMES

At the end of this course, the students will be able to:

COURSE OUTCOMES		KL
CO1	Explain the fundamentals of big data and data analytics and illustrate it.	K2
CO2	Discuss the Hadoop framework.	K2
CO3	Develop applications using exploratory data analysis and data manipulation tools.	K3
CO4	Analyze and interpret streaming data.	K4
CO5	Illustrate various applications of data analytics.	K2
CO6	Build applications for various domain.	K6

R.M.K GROUP OF INSTITUTIONS

CO – PO/PSO Mapping

R.M.K GROUP OF INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

6.CO – PO /PSO Mapping Matrix

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PS O1	PS O2	PS 03
CO1	3	2	1	2	2	-	-	1	2	1	-	3	3	3	3
CO2	3	2	1	2	3	-	-	1	2	2	-	2	3	3	3
CO3	3	2	2	3	3	-	-	1	2	1	2	3	3	3	3
CO4	3	3	2	3	3	-	-	1	2	1	-	3	3	3	3
CO5	3	3	2	3	1	-	-	1	2	2	2	3	3	3	3
CO6	3	3	3	3	2	-	-	1	2	2	2	3	3	3	3

R.M.K GROUP OF INSTITUTIONS

Lecture Plan

Unit - II

R.M.K GROUP OF INSTITUTIONS

7.LECTURE PLAN – Unit 2- HADOOP FRAMEWORK

Sl. No.	Topic	Number of Periods	Proposed Date	Actual Lecture Date	CO	Taxonomy Level	Mode of Delivery
1	Introducing Hadoop	1			2	K2	PPT/Chalk & Talk
2	RDBMS versus Hadoop	1			2	K2	PPT/Chalk & Talk
3	Hadoop Overview	1			2	K2	PPT/Chalk & Talk
4	HDFS (Hadoop Distributed File System)	2			2	K2	PPT/Chalk & Talk
5	Processing Data with Hadoop	1			2	K2	PPT/Chalk & Talk
6	Managing Resources and Applications with Hadoop YARN	1			2	K2	PPT/Chalk & Talk
7	Interacting with Hadoop Ecosystem	2			2	K2	PPT/Chalk & Talk
8	Working with Pandas data frames	3			2	K2	PPT
9	Basic plots using Matplotlib	3			2	K2	PPT

8. ACTIVITY BASED LEARNING

Hadoop Ecosystem Title: Real-Time Log Analysis and Visualization for Performance Monitoring

Imagine your team has been hired by a major E-commerce platform. The platform generates millions of log entries every hour. The management wants to understand:

•Which hours of the day experience the highest traffic (Peak Load)?
•Which specific URLs/Resources are failing (Status Code 404 or 500)?
•Which IP addresses are making an unusual number of requests (Potential DDoS attacks)?
•You are provided with a large dataset of raw .log files. Your task is to process this "Big Data" using the Hadoop Framework to generate actionable insights.

Activity Guidelines:

Team Formation: Form a group of **3 students** (e.g., Data Engineer, Script Developer, and Quality Analyst).

Step 1: Data Acquisition: Download a sample Apache Web Server log dataset (available on Kaggle or UCI Machine Learning Repository).

Step 2: MapReduce Logic:

Mapper: Parse the log line to extract the Timestamp, IP Address, or Status Code.

Reducer: Aggregate the counts based on the specific key (e.g., total hits per hour).

Step 3: Execution: Load the dataset into HDFS and execute your MapReduce Jar file.

Step 4: Peer Review & Mentorship: * Teams will present their **Execution Logic** and **Output Graphs**.

Each team will be reviewed by peers on: *Efficiency of the Map function*, *Handling of Null/Corrupt log lines*, and *Scalability*.

Useful link:

https://www.tutorialspoint.com/hadoop/hadoop_mapreduce.htm

UNIT-II
HADOOP FRAMEWORK

R.M.K GROUP OF INSTITUTIONS

9.LECTURE NOTES

1. INTRODUCING HADOOP

Today, Big Data seems to be the buzz word! Enterprises, the world over, are beginning to realize that there is a huge volume of untapped information before them in the form of structured, semi-structured and unstructured data. This varied variety of data is spread across the networks.

Let us look at few statistics to get an idea of the amount of data which gets generated every day, every minute and every second.

1. Every day:

(a) NYSE (New York Stock Exchange) generates 1.5 billion shares and trade data.

(b) Facebook stores 2.7 billion comments and Likes.

(c) Google processes about 24 petabytes of data.

2. Every

(a) Facebook users share nearly 2.5 million pieces of

(b) Twitter users tweet nearly 300,000

(c) Instagram users post nearly 220,000 new

(d) YouTube users upload 72 hours of new video content

(e) Apple users download nearly 50,000 apps.

(f) Email users send over 200 million messages.

(g) Amazon generates over $80,000 in online sales.

(h) Google receives over 4 million search queries.

3. Every second:

(a) Banking applications process more than 10,000 credit card transactions.

R.M.K GROUP OF INSTITUTIONS

1.1 Data : The Treasure Trove

1. Provides business advantages such as generating product recommendations, inventing new products, analyzing the market, and many, many more,..
2. Provides few early key indicators that can turn the fortune of business.
3. Provides room for precise analysis. If we have more data for analysis, then we have greater precision of analysis.

To process, analyze, and make sense of these different kinds of data, we need a system that scales and addresses the challenges shown in the below Figure 2.1.

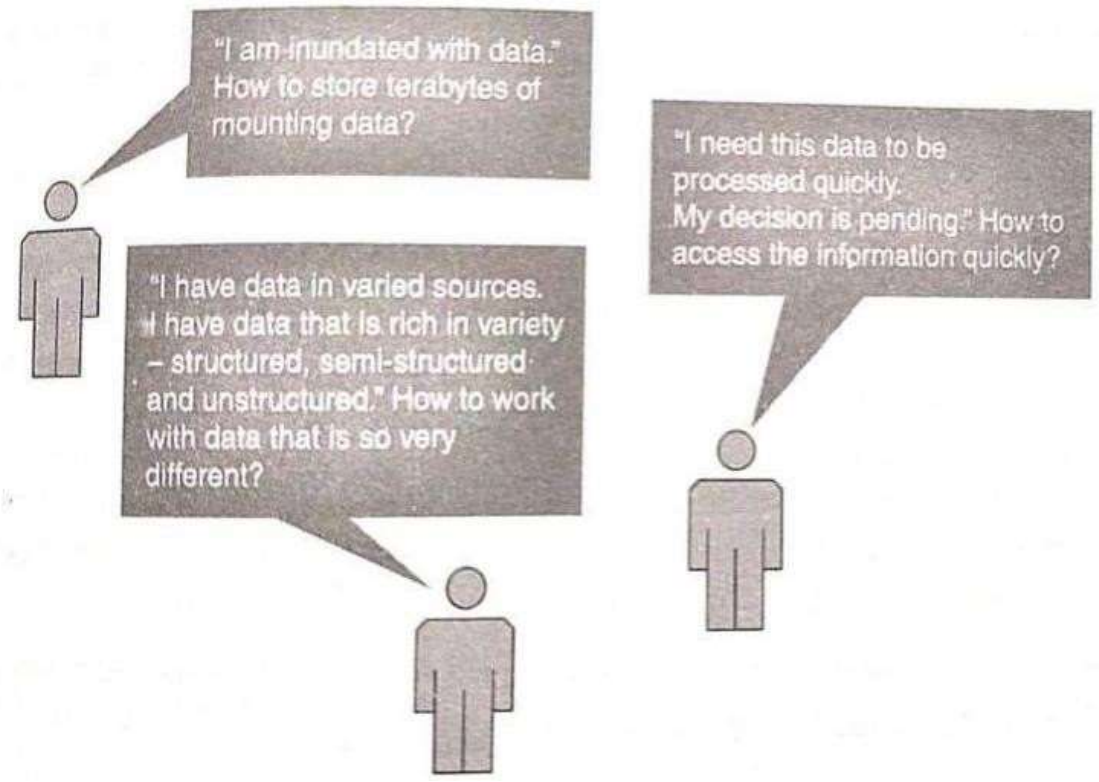


Figure 2.1 Challenges with big volume, Variety, and Velocity of data

- Hadoop is an open source software programming framework for storing a large amount of data and performing the computation. Its framework is based on Java programming with some native code in C and shell scripts.

2. RDBMS versus HADOOP

Table 2.1 describes the difference between RDBMS and Hadoop

Table 2.1 RDBMS versus Hadoop

PARAMETER	RDBMS	HADOOP
System	Relational Database Management System	Node Based Flat Structure
Data	Suitable for structured data	Suitable for structured, unstructured data. Supports variety of data formats in real time such as XML, JSON, text based flat file formats, etc.
Processing	OLTP	Analytical, Big Data Processing
Choice	When the data needs consistent relationship	Big Data processing, which does not require any consistent relationships between data.

R.M.K GROUP OF INSTITUTIONS

Processor	Needs expensive hardware or high-end processors to store huge volumes of data.	In a Hadoop Cluster, a node requires only a processor, a network card, and few hard drives.
Cost	Cost around $10,000 to $14,000 per terabytes of storage	Cost around $4,000 per terabytes of storage.

3. HADOOP OVERVIEW

Open-source software framework to store and process massive amounts of data in a distributed fashion on large clusters of commodity hardware. Basically, Hadoop accomplishes two tasks:

1. Massive data storage.
2. Faster data processing.

3.1 Key Aspects of Hadoop

The following are the key aspects of Hadoop:

- **Open source software:** It is free to download, use and contribute to.
- **Framework:** Means everything that you will need to develop and execute and application is provided-programs, tools,etc.
- **Distributed:** Divides and stores data across multiple computers. Computation/Processing is done in parallel across multiple connected nodes.
- **Massive storage:** Stores colossal amounts of data across nodes of low cost commodity hardware.
- **Faster processing:** Large amounts of data is processed in parallel, yielding quick response.

R.M.K GROUP OF INSTITUTIONS

3.2 Hadoop Components

The following figure 2.2 depicts the Hadoop components.

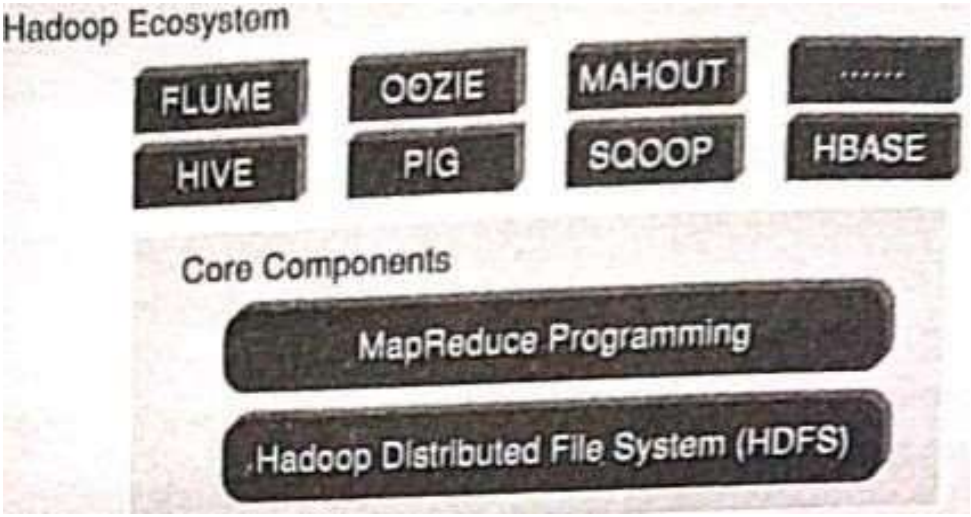


Figure 2.2 Hadoop Components

Hadoop Core Components

1. HDFS:

(a) Storage

(b) Distributes data across several

(c) Natively redundant.

2. MapReduce:

(a) Computational framework.

(b) Splits a task across multiple nodes.

(c) Processes data in parallel.

Hadoop Ecosystem:

Hadoop Ecosystem are support projects to enhance the functionality of Hadoop Core Components.

R.M.K GROUP OF INSTITUTIONS

The Eco Projects are as follows:

1. HIVE
2. PIG
3. SQOOP
4. HBASE
5. FLUME
6. OOZIE
7. MAHOUT

3.3 Hadoop Conceptual Layer

It is conceptually divided into **Data Storage Layer** which stores huge volumes of data and **Data Processing Layer** which processes data in parallel to extract, richer and meaningful insights from data (Figure 2.3)

3.4 High-Level Architecture of Hadoop

Hadoop is a distributed **Master-Slave** Architecture. Master node is known as **NameNode** and slave nodes are known as **DataNodes**. Figure 2.4 depicts the Master-Slave Architecture of Hadoop Framework.

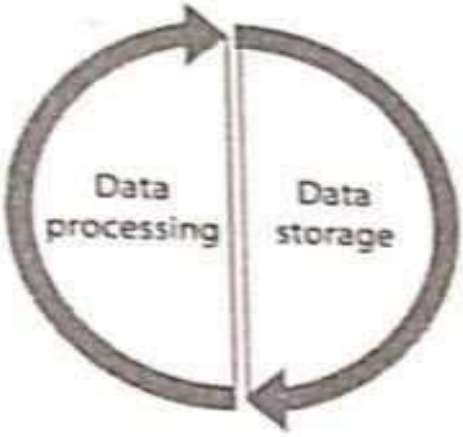


Figure 2.3 Hadoop conceptual layer

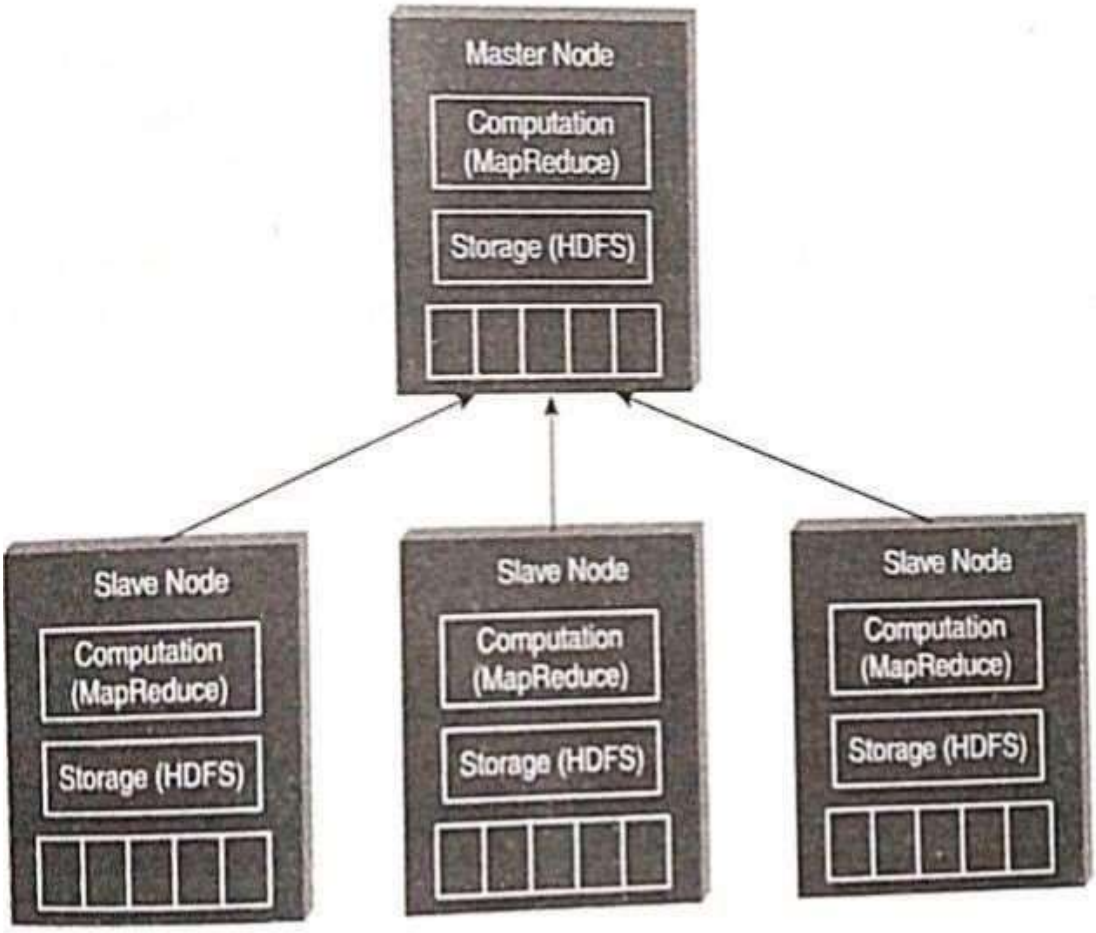


Figure 2.4 Hadoop high-level architecture

Let us look at the key components of the Master node

1. Master HDFS : Its main responsibility is partitioning the data storage across the slave nodes. It also keeps track of locations of data on DataNodes.

2. **Master MapReduce :** It decides and schedules computation task on slave nodes.

4. HDFS (HADOOP DISTRIBUTED FILE SYSTEM)

Some key Points of Hadoop Distributed File System are as follows:

1. Storage component of Hadoop.
2. Distributed File System.

3. Modeled after Google File System.
4. Optimized for high throughput (HDFS leverages large block size and moves computation where data is stored).
5. We can replicate a file for a configured number of times, which is tolerant in terms of both software and hardware.
6. Re-replicates data blocks automatically on nodes that have failed.
7. We can realize the power of HDFS when we perform read or write on large files (gigabytes and larger).
8. Sits on top of native file system such as ext3 and ext4, which is described in Figure 2.5.

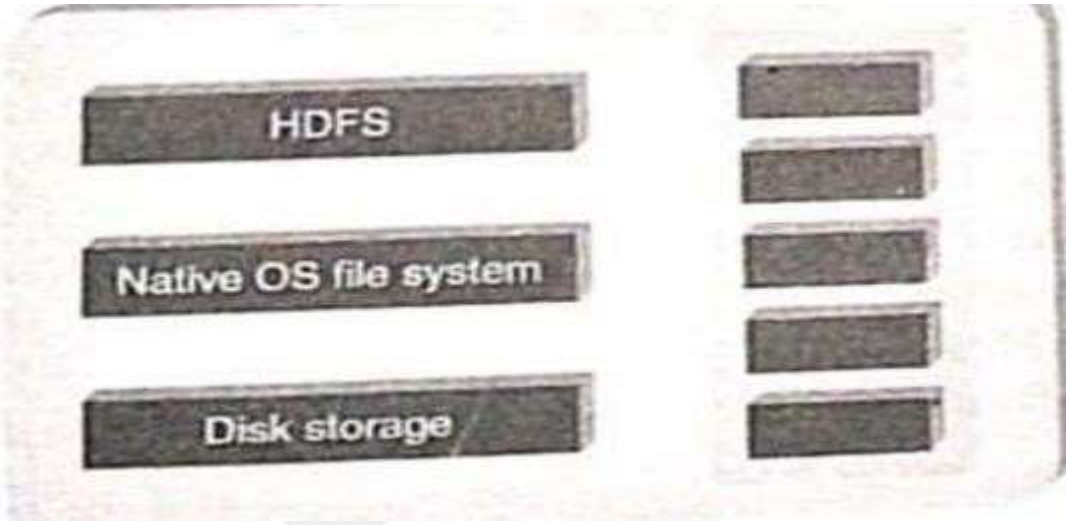


Figure 2.5 Hadoop Distribute File System

Figure 2.6 describes important key points of HDFS. Figure 2.7 describes Hadoop Distributed File system architecture. Client Application interacts with NameNode for metadata related activities and communicates with DataNodes to read and write files. DataNodes converse with each other for pipeline reads and writes.

Hadoop Distributed – Key Points		
Block structured File	Default Replication Factor:3	Default Block Size:64 MB

Figure 2.6 HDFS-Key Points

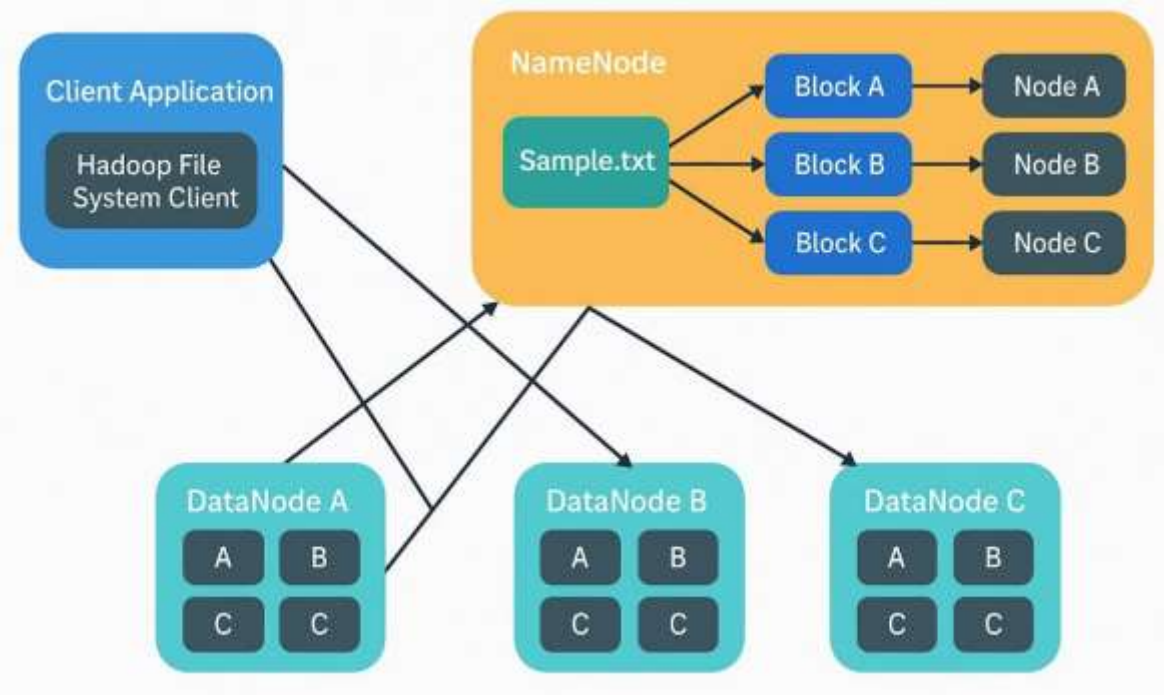


Figure 2.7 Hadoop Distributed file system Architecture

Let us assume that the file "Sample.txt" is of size 192 MB. As per the default data block size (64 MB), it will be split into three blocks and replicated across the nodes on the cluster based on the default replication factor.

1. HDFS Daemons

1. NameNode

- HDFS breaks a large file into smaller pieces called **blocks**. NameNode uses a **rack ID** to identify DataNodes in the rack.
- A rack is a collection of DataNodes within the cluster. NameNode keeps tracks of blocks of a file as it is placed on various DataNodes.
- NameNode manages file-related operations such as read, write, create and delete. Its main job is managing the **File System Namespace**.
- A file system namespace is collection of files in the cluster. NameNode stores HDFS namespace.
- File system namespace includes mapping of blocks to file, file properties and is stored in a file called **FsImage**.

R.M.K GROUP OF INSTITUTIONS

- NameNode uses an EditLog (transaction log) to record every transaction that happens to the file system metadata. Figure 2 . 8 .
- When NameNode starts up, it reads FsImage and EditLog from disk and applies all transactions from the EditLog to in-memory representation of the FsImage.
- Then it flushes out new version of Fslmage on disk and truncates the old EditLog because the changes are updated in the Fslmage. There is a single NameNode per cluster.

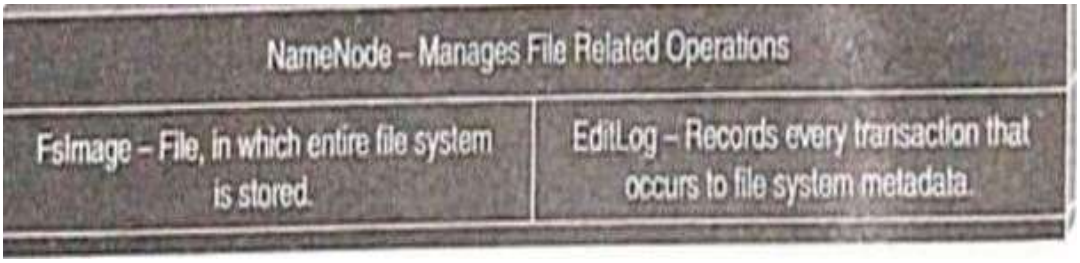


Figure 2.8 NameNode

4.1.2 DataNode

- There are multiple DataNodes per cluster. During Pipeline read and write DataNodes communicate with each other. A DataNode also continuously sends "heartbeat" message to NameNode to ensure the connectivity between the NameNode and DataNode.
- In case there is no heartbeat from a DataNode, the NameNode replicates that DataNode within the cluster and keeps on running as if nothing had happened.

4.1.3 Secondary NameNode

- The Secondary NameNode takes a snapshot of HDFS metadata at intervals specified in the Hadoop configuration. Since the memory requirements of Secondary NameNode are the same as NameNode, it is better to run NameNode and Secondary NameNode on different machines.
- In case of failure of the NameNode, the Secondary NameNode can be configured manually to bring up the cluster.
- However, the Secondary NameNode does not record any real-time changes that happen to the HDFS metadata.

R.M.K GROUP OF INSTITUTIONS

4.2 Anatomy of File Read

Figure 2.9 describes the anatomy of File Read

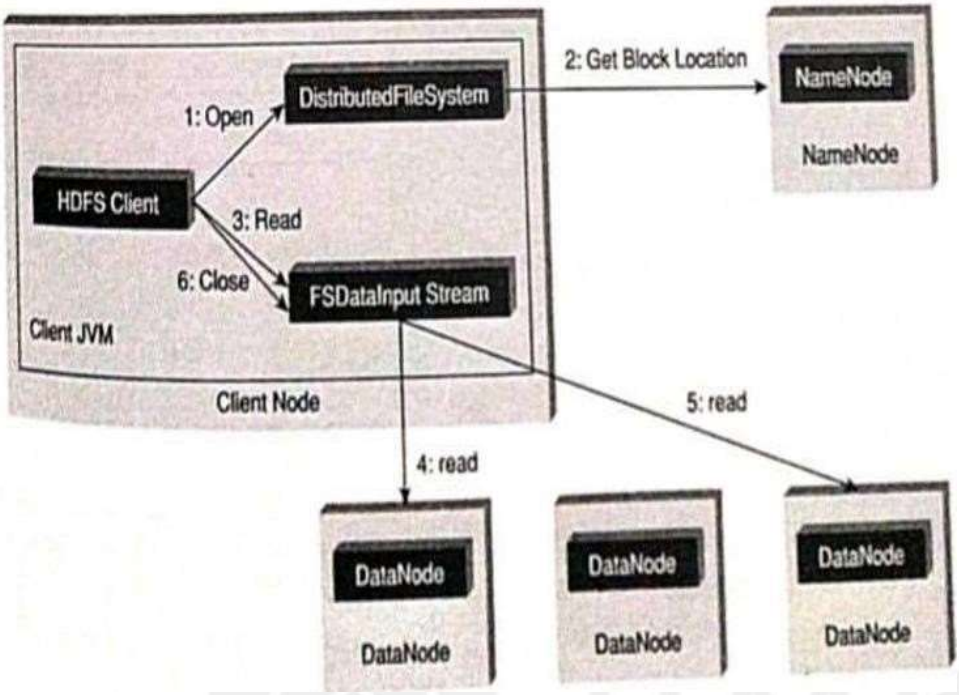


Figure 2.9 File Read

The steps involved in the File Read are as follows:

1. The client opens the file that it wishes to read from by calling open() on the DistributedFileSystem.
2. DistributedFileSystem communicates with the NameNode to get the location of data blocks. NameNode returns with the addresses of the DataNodes that the data blocks are stored on. Subsequent to this, the DistributedFileSystem returns an FSDataInputStream to client to read from the file.
3. Client then calls read() on the stream DFSInputStream, which has addresses of the DataNodes for the first few blocks of the file, connects to the closest DataNode for the first block in the file.
4. Client calls read() repeatedly to stream the data from the DataNode.
5. When end of the block is reached, DFSInputStream closes the connection with the DataNode. It repeats the steps to find the best DataNode for the next block and subsequent blocks.
6. When the client completes the reading of the file, it calls close() on the FSDataInputStream to close the connection.

R.M.K GROUP OF INSTITUTIONS

4.3 Anatomy of File Write

Figure 2.10 describes the anatomy of File Write. The steps involved in anatomy of File Write are as follows:

1. The client calls create() on DistributedFileSystem to create a file.
2. An RPC call to the NameNode happens through the DistributedFileSystem to create a new file. The NameNode performs various checks to create a new file (checks whether such a file exists or not).Initially, the NameNode creates a file without associating any data blocks to the file. The DistributedFileSystem returns an FSDataOutputStream to the client to perform write.
3. As the client writes data, data is split into packets by DFSOutputStream, which is then written to an internal queue, called data queue. DataStreamer consumes the data queue. The DataStreamer requests the NameNode to allocate new blocks by selecting a list of suitable DataNodes to store replicas. This list of DataNodes makes a pipeline. Here, we will go with the default replication factor of three, so there will be three nodes in the pipeline for the first block.
4. DataStreamer streams the packets to the first DataNode in the pipeline. It stores packet and forwards it to the second DataNode in the pipeline.
5. In the same way, the second DataNode stores the packet and forwards it to the third DataNode in the pipeline.
6. In addition to the internal queue, DFSOutputStream also manages an "Ack queue" of packets that are waiting for the acknowledgement by DataNodes. A packet is removed from the "Ack queue" only if it is acknowledged by all the DataNodes in the pipeline.
7. When the client finishes writing the file, it calls close() on the stream.
8. This flushes all the remaining packets to the DataNode pipeline and waits for relevant acknowledgments before communicating with the NameNode to inform the client that the creation of the file is complete.

R.M.K GROUP OF INSTITUTIONS

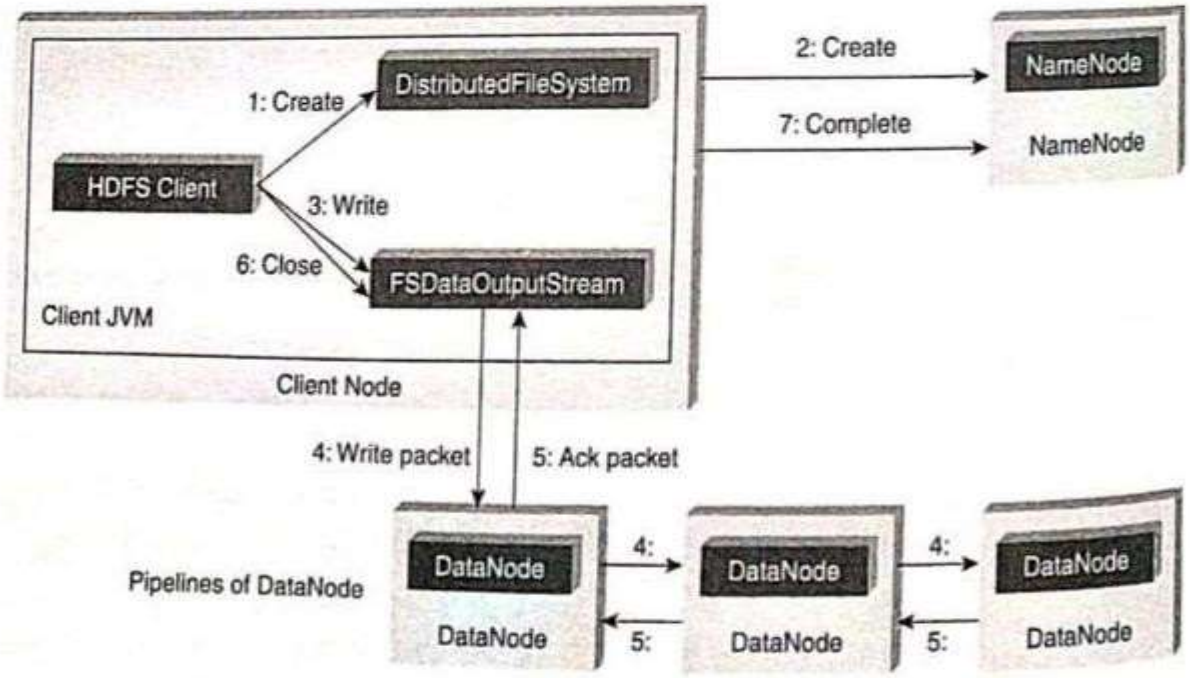


Figure 2.10 File Write

4.4 Replica Placement Strategy

Hadoop Default Replica Placement Strategy

As per the Hadoop Replica Placement Strategy, first replica is placed on the same node as the client. Then it places second replica on a node that is present on different rack. It places the third replica on the same rack as second, but on a different node in the rack. Once replica locations have been set, a pipeline is built. This strategy provides good reliability. Figure 2.11 describes the typical replica pipeline.

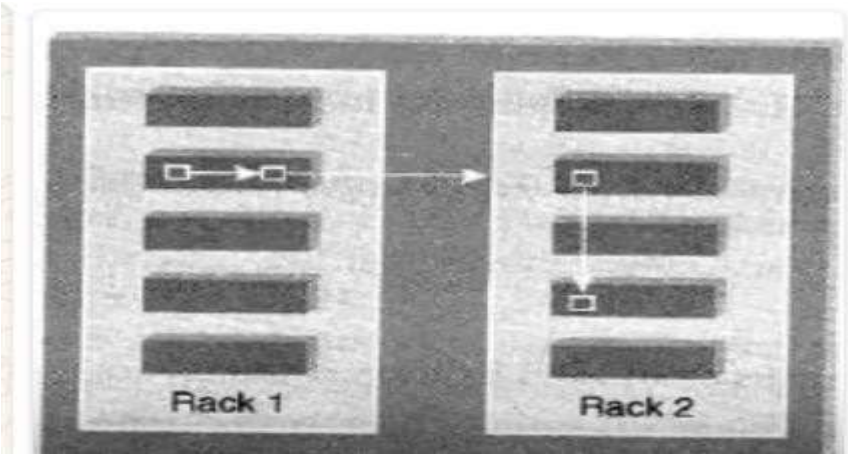


Figure 2.11 Replica Placement Strategy

R.M.K GROUP OF INSTITUTIONS

4.5 Working with HDFS Commands

- **Objective:** To get the list of directories and files at the root of HDFS.

Act:

hadoop fs -ls/

- **Objective:** To get the list of complete directories and files of HDFS.

Act:

hadoop fs -ls -R/

- **Objective:** To create a directory (say, sample) in HDFS.

Act:

hadoop fs -mkdir / sample

- **Objective:** To copy a file from local file system to **HDFS.**

Act:

hadoop fs -put / root/sample/test.txt /sample/test.txt

- **Objective:** To copy a file from HDFS to local file system

Act:

hadoop fs -get/sample/test.txt /root/sample/testsample.txt

- **Objective:** Tocopy a file from local file system to HDFS via copyFromLocal

hadoop fs -copyFromLocal /root/sample/test.txt /sample/testsample.txt

R.M.K GROUP OF INSTITUTIONS

➢ **Objective:** To copy a file from Hadoop file system to local file system via copyToLocal command.

Act:

hadoop fs -copyToLocal /sample/test.txt /root/sample/testsample1.txt

➢ **Objective:** To display the contents of an HDFS file on console.

Act:

hadoop fs -cat /sample/test.txt

➢ **Objective:** To copy a file from one directory to another on HDFS.

Act:

hadoop fs -cp /sample/test.txt /sample1

➢ **Objective:** To remove a directory from HDFS.

Act: hadoop fs-rm-r/sample1

4.6 Special Features of HDFS

1. Data Replication:

There is absolutely no need for a client application to track all blocks. It directs the client to the nearest replica to ensure high performance.

2. Data Pipeline:

A client application writes a block to the first DataNode in the pipeline. Then this DataNode takes over and forwards the data to the next node in the pipeline. This process continues for all the data blocks, and subsequently all the replicas are written to the disk.

R.M.K GROUP OF INSTITUTIONS

5. Processing Data with Hadoop

- MapReduce Programming is a software framework. MapReduce Programming helps us to process massive amounts of data in parallel.

- In MapReduce Programming, the input dataset is split into independent chunks. **Map tasks** process these independent chunks completely in a parallel manner.

- The output produced by the map tasks serves as intermediate data and is stored on the local disk of that server. The output of the mappers are automatically shuffled and sorted by the framework.

- MapReduce Framework sorts the output based on **keys**. This sorted Output becomes the input to the **reduce tasks**. Reduce task provides reduced output by combining the output of the various mappers.

- Job inputs and outputs are stored in a file system. MapReduce framework also takes care of the other tasks such as scheduling, monitoring, re-executing failed tasks, etc.

- Hadoop Distributed File System and MapReduce Framework run on the same set of nodes. This configuration allows effective scheduling of tasks on the nodes where data is present (**Data Locality**). This in turn results in very high throughput.

- There are two daemons associated with MapReduce Programming. A single master **JobTracker** per cluster and one slave **TaskTracker** per cluster-node. The JobTracker is responsible for scheduling tasks to the TaskTrackers, monitoring the task, and re-executing the task just in case the TaskTracker fails. The TaskTracker executes the task. Figure 2.12

R.M.K GROUP OF INSTITUTIONS

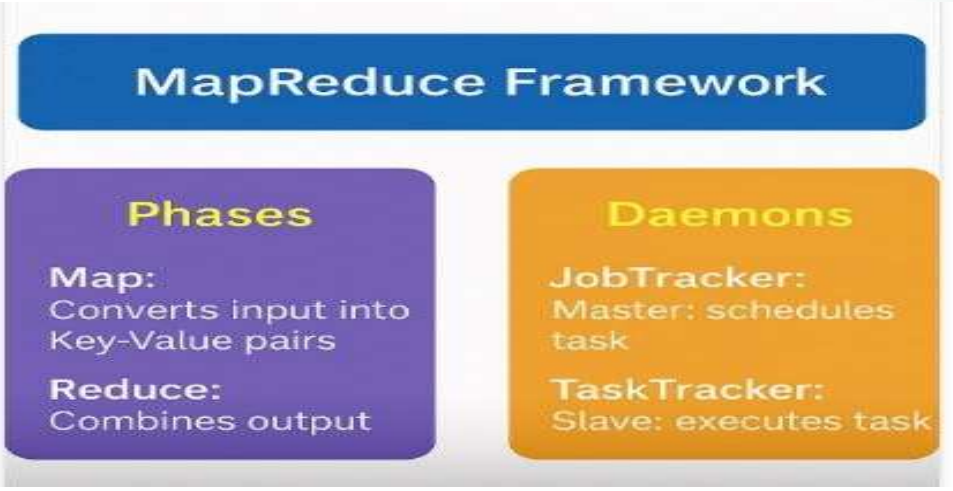


Figure 2.12 MapReduce Programming phases and Daemons

- The MapReduce functions and input/output locations are implemented via the MapReduce applications. These applications use suitable interfaces to construct the job.

- The application and the job parameters together are known as **job configuration**. Hadoop **job client** submits job (jar/executable, etc.) to the JobTracker. Then it is the responsibility of JobTracker to schedule tasks to the slaves. In addition to scheduling, it also monitors the task and provides status information to the job- client.

1. MapReduce Daemons

1. JobTracker:

- It provides connectivity between Hadoop and our application. When we submit code to cluster, JobTracker creates the execution plan by deciding which task to assign to which node. It also monitors all the running tasks.

- When a task fails, it automatically re-schedules the task to a different node after a predefined number of retries. JobTracker is a master daemon responsible for executing overall MapReduce job. There is a single JobTracker per Hadoop cluster.

5.1.2 TaskTracker:

- This daemon is responsible for executing individual tasks that is assigned by the JobTracker. There is a single TaskTracker per slave and spawns multiple Java Virtual Machines (JVMs) to handle multiple map or reduce tasks in parallel.

- TaskTracker continuously sends heartbeat message to JobTracker. When the JobTracker fails to receive a heartbeat from a TaskTracker, the JobTracker assumes that the TaskTracker has failed and resubmits the task to another available node in the clusters.

- Once the client submits a job to the JobTracker, it partitions and assigns diverse MapReduce tasks for each TaskTracker in the cluster. Figure 2.13 depicts JobTracker and TaskTracker interaction.

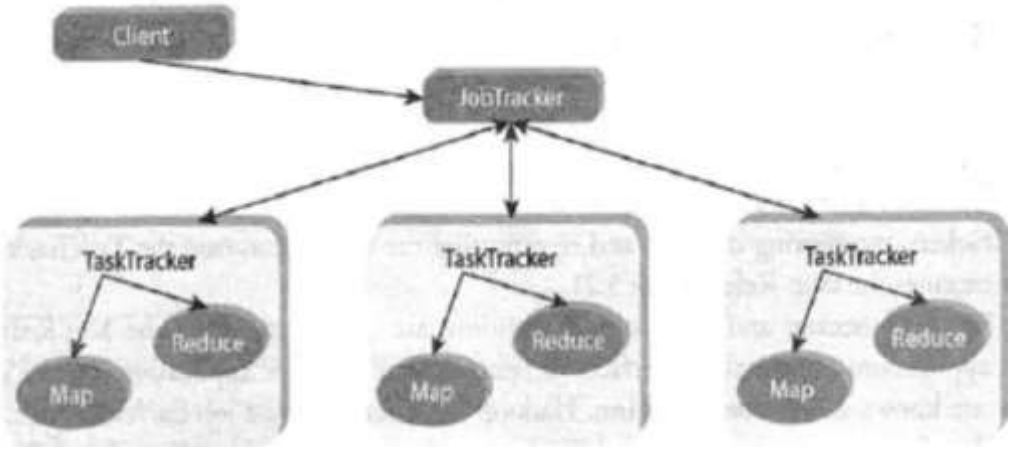


Figure 2.13 JobTracker and TaskTracker interaction

5.2 How Does MapReduce Work?

❖ MapReduce divides a data analysis task into two parts - **map** and **reduce**. Figure 2.14 depicts how the MapReduce Programming works.

- In this example, there are two mappers and one reducer. Each mapper works on the partial dataset that is stored on that node and the reducer combines the output from the mappers to produce the reduced result set.

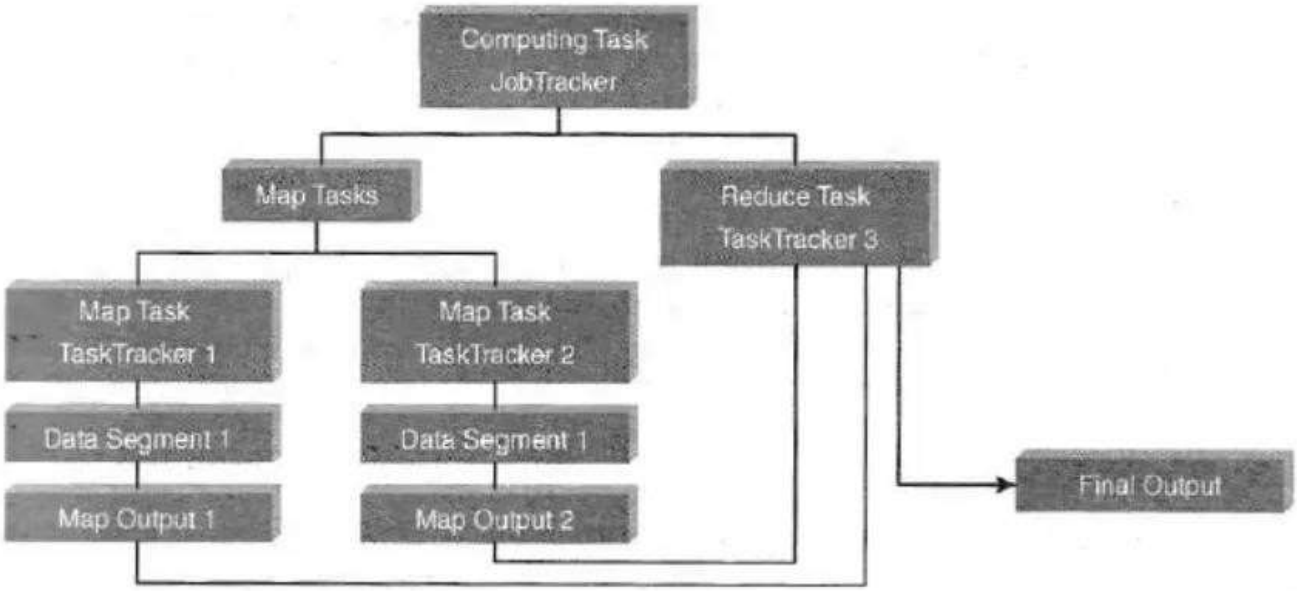


Figure 2.14 MapReduce Programming workflow

- Figure 2.15 describes the working model of MapReduce Programming. The following steps describe how MapReduce performs its task.

1. First, the input dataset is split into multiple pieces of data (several small subsets).
2. Next, the framework creates a master and several workers processes and executes the worker processes remotely.
3. Several map tasks work simultaneously and read pieces of data that were assigned to each map task. The map worker uses the map function to extract only those data that are present on their server and generates key/value pair for the extracted data.
4. Map worker uses partitioner function to divide the data into regions. Partitioner decides which reducer should get the output of the specified mapper

R.M.K GROUP OF INSTITUTIONS

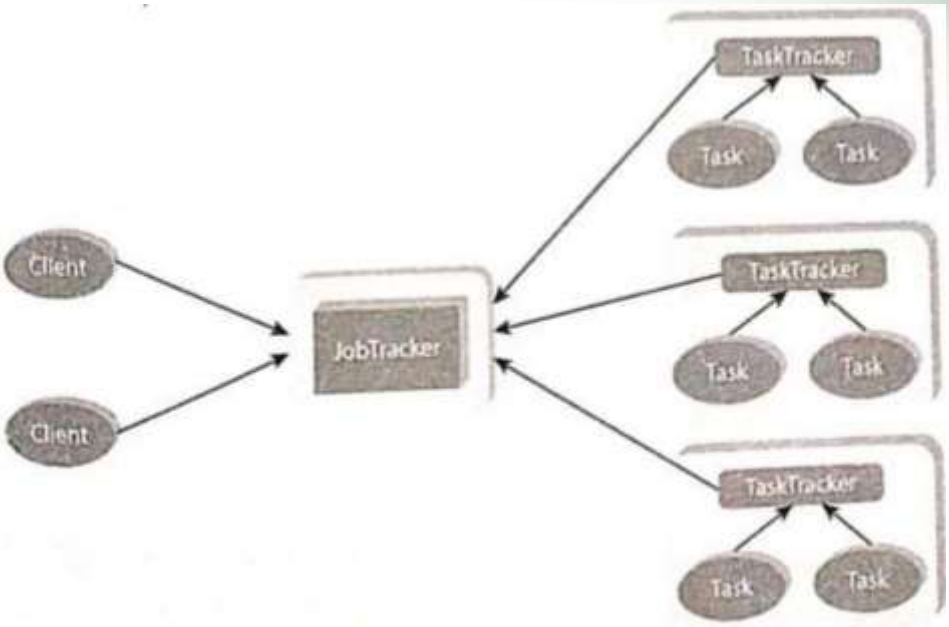


Figure 2.15 MapReduce programming architecture

5. When the map workers complete their work, the master instructs the reduce workers to begin their work. The reduce workers in turn contact the map workers to get the key/value data for their partition. The data thus received is shuffled and sorted as per keys.

6. Then it calls reduce function for every unique key. This function writes the output to the file.

7. When all the reduce workers complete their work, the master transfers the control to the user program.

5.3 MapReduce Example

The famous example for MapReduce Programming is **Word Count**. For example, consider we need to count the occurrences of similar words across 50 files. We can achieve this using MapReduce Programming. Refer Figure 5.16.

R.M.K GROUP OF INSTITUTIONS

Word Count MapReduce Programming using Java

The MapReduce Programming requires three things.

1. **Driver Class:** This class specifies **Job Configuration** details.
2. **Mapper Class:** This class overrides the **Map Function** based on the problem statement.
3. **Reducer Class:** This class overrides the **Reduce Function** based on the problem statement.

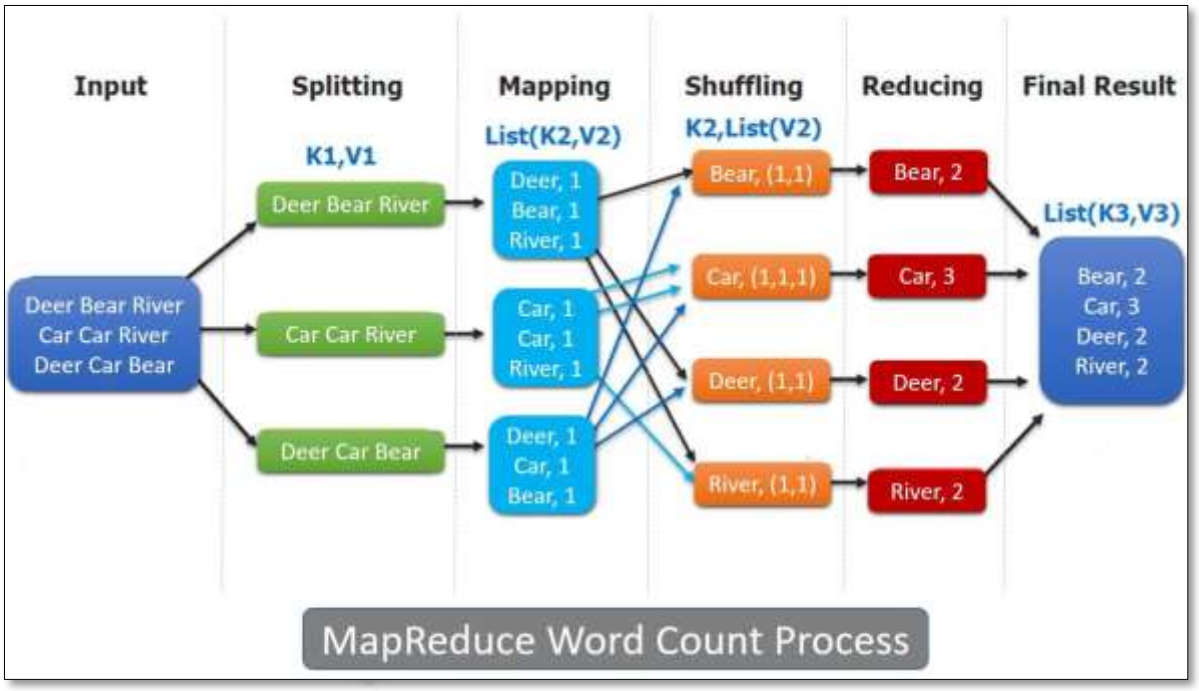


Figure 2.16 Wordcount Example

❖ Wordcounter.java: Driver Program

package com.app;

import java.io.IOException;

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class WordCounter {

public static void main (String [] args) throws IOException,
InterruptedException, ClassNotFoundException {
        Job job = new Job ();
        job.setJobName ("wordcounter");
        job.setJarByClass (WordCounter.class);
        job.setMapperClass (WordCounterMap.class);
        job.setReducerClass (WordCounterRed.class);
        job.setOutputKeyClass (Text.class);
        job.setOutputValueClass (IntWritable.class);

        FileInputFormat.addInputPath (job, new Path ("/sample/word.
txt"));
        FileOutputFormat.setOutputPath (job, new Path ("/sample/
wordcount"));
        System.exit (job.waitForCompletion (true)? 0: 1);

    }
}
```

❖ WordCounterMap.java: Map Class

```
package com.app;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

Public class WordCounterMap extends Mapper<LongWritable, Text, Text,
IntWritable> {
    @Override

    protected void map (LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

        String [] words=value.toString ().split (",");
        for (String word: words) {
            context.write (new Text (word), new IntWritable (1));
        }
    }
}
```

❖ WordCountReduce.java: Reduce Class

```
package com.infosys;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
```

```
public class WordCounterRed extends Reducer<Text, IntWritable, Text,
IntWritable>{

    @Override
    protected void reduce(Text word, Iterable<IntWritable> values,
Context context)
            throws IOException, InterruptedException {
        Integer count = 0;
        for(IntWritable val: values){
            count += val.get();
        }
        context.write(word, new IntWritable(count));
    }
}
```

Table 2.2 describes differences between SQL and MapReduce.

Table 2.2 SQL versus MapReduce

	SQL	MapReduce
Access	Interactive and Batch	Batch
Structure	Static	Dynamic
Updates	Read and write many times	Write once, read many times
Integrity	High	Low
Scalability	Nonlinear	Linear

6. MANAGING RESOURCES AND APPLICATIONS WITH HADOOP YARN (YET ANOTHER RESOURCE NEGOTIATOR)

- Apache Hadoop YARN is a sub-project of Hadoop 2.x.
- Hadoop 2.x is YARN-based architecture. It is a general processing platform.

- YARN is not constrained to MapReduce only. We can run multiple applications in Hadoop 2.x in which all applications share a common resource management.
- Now Hadoop can be used for various types of processing such as Batch, Interactive, Online, Streaming, Graph, and others.

6.1 Limitations of Hadoop 1.0 Architecture

In Hadoop 1.0, HDFS and MapReduce are Core Components, while other components are built around the core.

1. Single NameNode is responsible for managing entire namespace for Hadoop Cluster.
2. It has a restricted processing model which is suitable for batch-oriented MapReduce jobs.
3. Hadoop MapReduce is not suitable for interactive analysis.
4. Hadoop 1.0 is not suitable for machine learning algorithms, graphs, and other memory intensive algorithms.
5. **MapReduce** is responsible for **cluster resource management and data processing**.

- In this Architecture, **map slots might be "full", while the reduce slots are empty and vice versa**.
- This causes **resource utilization issues**. This needs to be improved for proper resource utilization.

R.M.K GROUP OF INSTITUTIONS

6.2 HDFS Limitation

- NameNode saves all its file metadata in main memory. Although the main memory today is not as small and as expensive as it used to be two decades ago, still there is a limit on the number of objects that one can have in the memory on a single NameNode.
- The NameNode can quickly become overwhelmed with load on the system increasing.
- In Hadoop 2.x, this is resolved with the help of **HDFS Federation**.

6.3 Hadoop 2: HDFS

HDFS 2 consists of two major components:

(a) namespace

(b) blocks storage service.

Namespace service takes care of file-related operations, such as creating files, modifying files, and directories. The block storage service handles data node cluster management, replication.

HDFS 2 Features

1. Horizontal scalability
2. High availability

- HDFS Federation uses multiple independent NameNodes for horizontal scalability.
- NameNodes are independent of each other. It means, NameNodes does not need any coordination with each other.
- The DataNodes are common storage for blocks and shared by all NameNodes.
- All DataNodes in the cluster registers with each NameNode in the cluster.

R.M.K GROUP OF INSTITUTIONS

- High availability of NameNode is obtained with the help of **Passive Standby NameNode.** In Hadoop 2.x, Active-passive NameNode handles failover automatically.
- All namespace edits are recorded to a shared NFS storage and there is a

single writer at any point of time. Passive NameNode reads edits from shared storage and keeps updated metadata information.

- In case of Active NameNode failure, Passive NameNode becomes an Active NameNode automatically. Then it starts writing to the shared storage.
- Figure 2.17 describes the Active-Passive NameNode interaction.

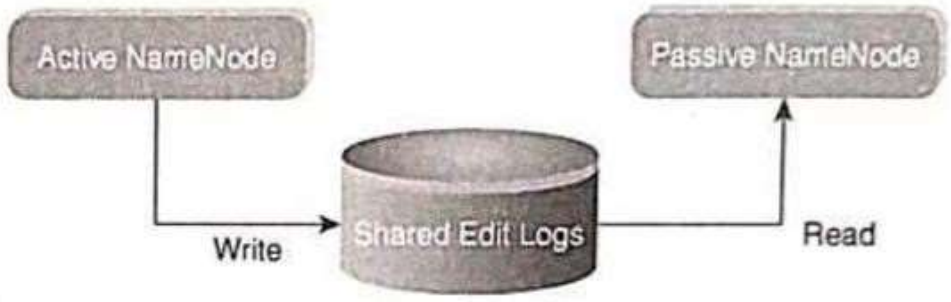


Figure 2.17 depicts Hadoop 1.0 and Hadoop 2.0 architecture.

R.M.K GROUP OF INSTITUTIONS

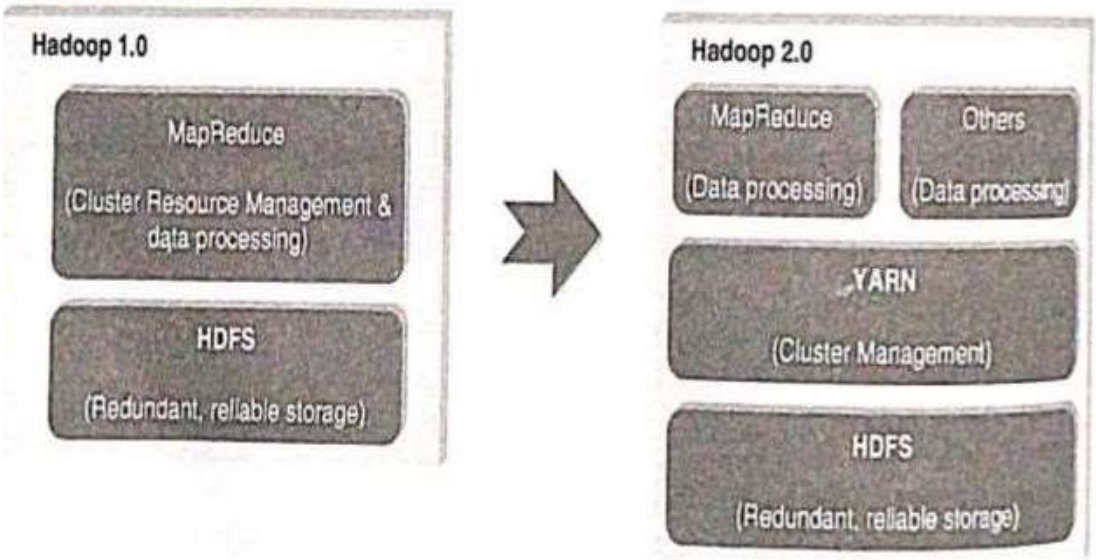


Figure 2.18 Hadoop 1.x versus Hadoop

6.4 Hadoop 2 YARN: Taking Hadoop beyond Batch

- YARN helps us to store all data in one place.
- We can interact in multiple ways to get predictable performance and quality of services.
- This was originally architected by Yahoo. Refer Figure 5.28.

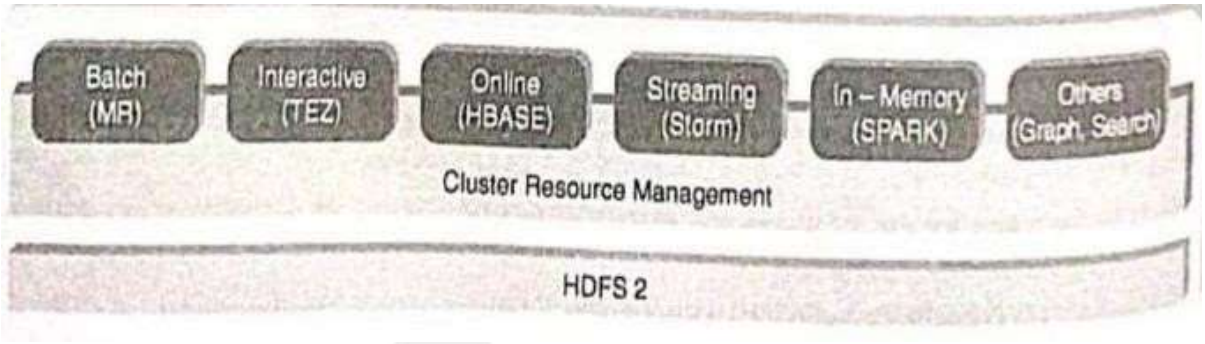


Figure 2.19 Hadoop YARN

6.4.1 Fundamental Idea

The fundamental idea behind this architecture is splitting the JobTracker responsibility of resource management and Job scheduling / Monitoring into separate daemons.

Daemons that are part of YARN Architecture are described below.

1. **A Global ResourceManager:**

Its main responsibility is to distribute resources among various applications in the system. It has two main components:

(a) **Scheduler:** The pluggable scheduler of ResourceManager decides allocation of resources to various running applications. The scheduler is just that, a pure scheduler, meaning it does NOT monitor or track the status of the application.

(b) **ApplicationManager:** ApplicationManager does the following:

- Accepting job submissions.
- Negotiating resources (container) for executing the application specific ApplicationMaster.
- Restarting the ApplicationMaster in case of failure.

R.M.K GROUP OF INSTITUTIONS

2. NodeManager:

- This is a per-machine slave daemon.
- NodeManager responsibility is launching the application containers for application execution.
- NodeManager monitors the resource usage such as memory, CPU, disk, network, etc. It then reports the usage of resources to the global ResourceManager.

3. Per-application ApplicationMaster:

- This is an application-specific
- Its responsibility is to negotiate required resources for execution from the ResourceManager
- It works along with the NodeManager for executing and monitoring component tasks.

6.4.2 Basic Concepts

Application:

1. Application is a job submitted to the framework.
2. Example- **MapReduce Job**.

Container:

1. Basic unit of allocation.
2. Fine-grained resource allocation across multiple resource types (Memory, CPU, disk, network, etc.)

(a) container_0 = 2GB, 1 CPU

(b) container_1 = 1GB, 6 CPU

3. Replaces the fixed map/reduce slots.

R.M.K GROUP OF INSTITUTIONS

YARN Architecture:

Figure 2.20 depicts YARN architecture.

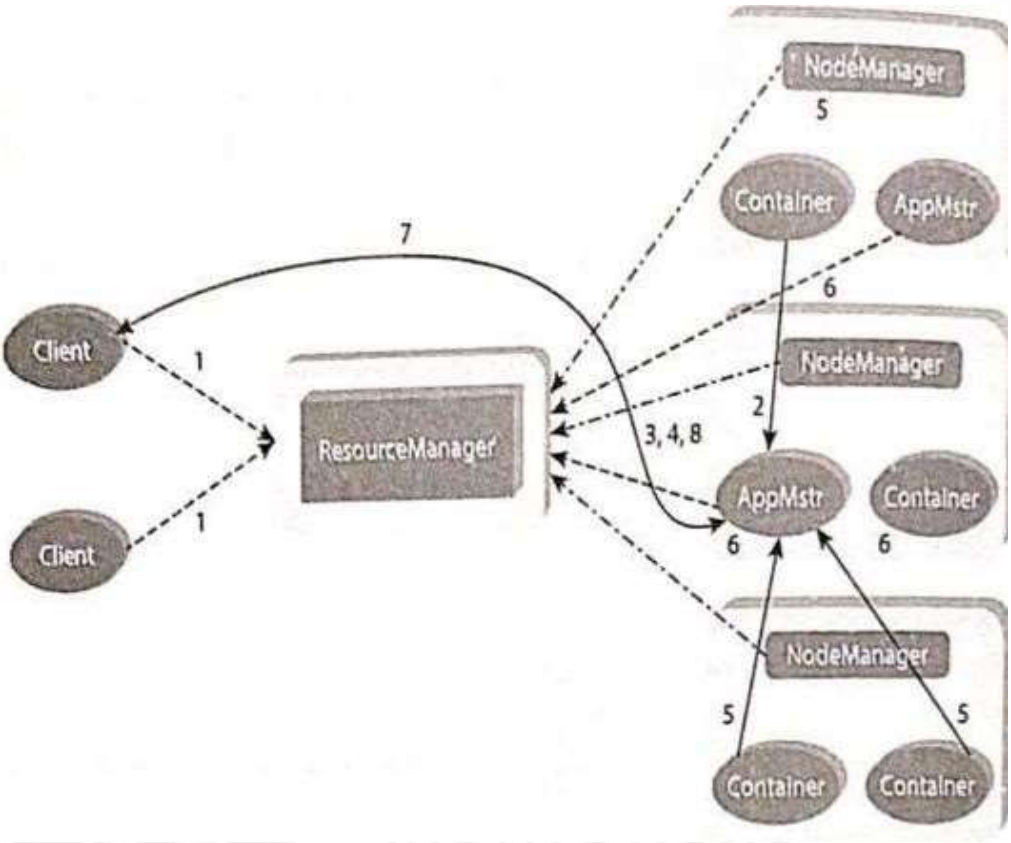


Figure 2.20 YARN architecture

The steps involved in YARN architecture are as follows:

1. A client program submits the application which includes the necessary specifications to launch the application – specific ApplicationMaster itself.
2. The ResourceManager launches the ApplicationMaster by assigning some container.
3. The ApplicationMaster, on boot-up, registers with the ResourceManager. This helps the client program to query the ResourceManager directly for the details.
4. During the normal course, ApplicationMaster negotiates appropriate resource containers via the resource-request protocol.

5. On successful container allocations, the ApplicationMaster launches the container by providing the container launch specification to the NodeManager.

6. The NodeManager executes the application code and provides necessary information such as progress status, etc. to its ApplicationMaster via an application-specific protocol.

7. During the application execution, the client that submitted the job directly communicates with the ApplicationMaster to get status, progress updates, etc. via an application-specific protocol.

9. Once the application has been processed completely, ApplicationMaster deregisters with the ResourceManager and shuts down, allowing its own container to be repurposed.

7. INTERACTING WITH HADOOP ECOSYSTEM

1. Pig

- Pig is a data flow system for Hadoop. It uses Pig Latin to specify data flow.
- Pig is an alternative to MapReduce Programming. It abstracts some details and allows us to focus on data processing.
- It consists of two components.
  1. Pig Latin: The data processing language.
  2. Compiler: To translate Pig Latin to MapReduce Programming.
- Figure 2.21 depicts the Pig in the Hadoop ecosystem.

R.M.K GROUP OF INSTITUTIONS

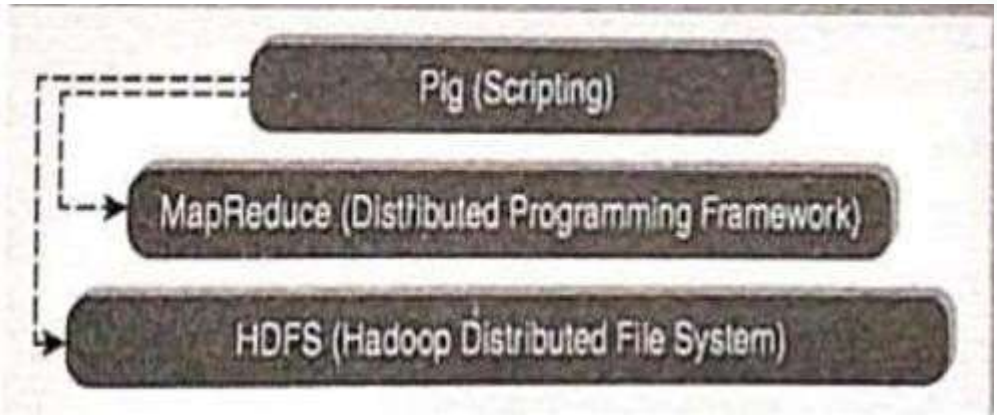


Figure 2.21 Pig in the Hadoop Ecosystem

7.2 Hive

- Hive is a Data Warehousing Layer on top of
- Analysis and queries can be done using an SQL-like
- Hive can be used to do ad-hoc queries, summarization, and data analysis.

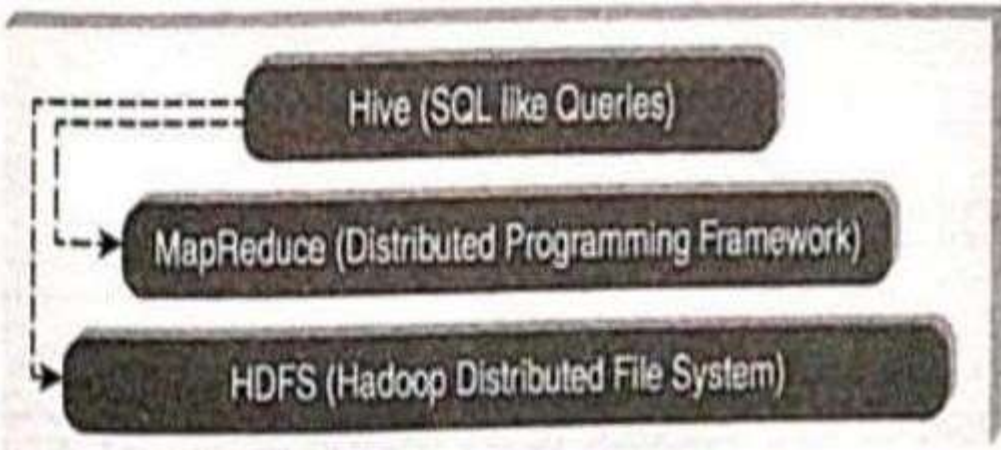


Figure 2.22 Hive in the Hadoop Ecosystem

7.3 Sqoop

- Sqoop is a tool which helps to transfer data between Hadoop and Relational Databases.
- With the help of Sqoop, we can import data from RDBMS to HDFS and vice-versa. Figure 2.23 depicts the Sqoop in Hadoop ecosystem.

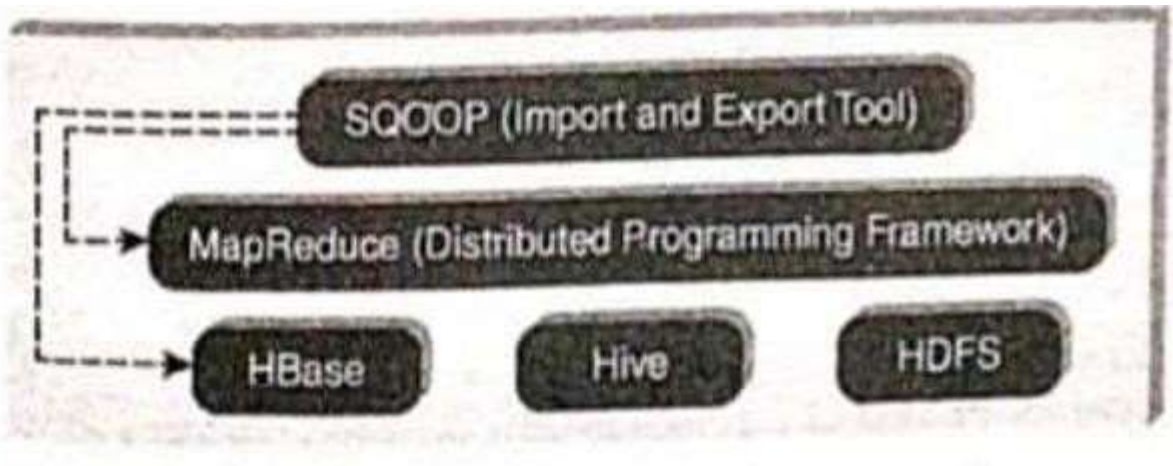


Figure 2.23 Sqoop in Hadoop ecosystem

7.4 HBase

- HBase is a NoSQL database for Hadoop.
- HBase is column-oriented NoSQL
- HBase is used to store **billions of rows and millions of columns**.
- HBase provides random read/write operation. It also supports record level updates which is not possible using HDFS. HBase sits on top of HDFS.

Figure 2.24 depicts the HBase in Hadoop ecosystem.

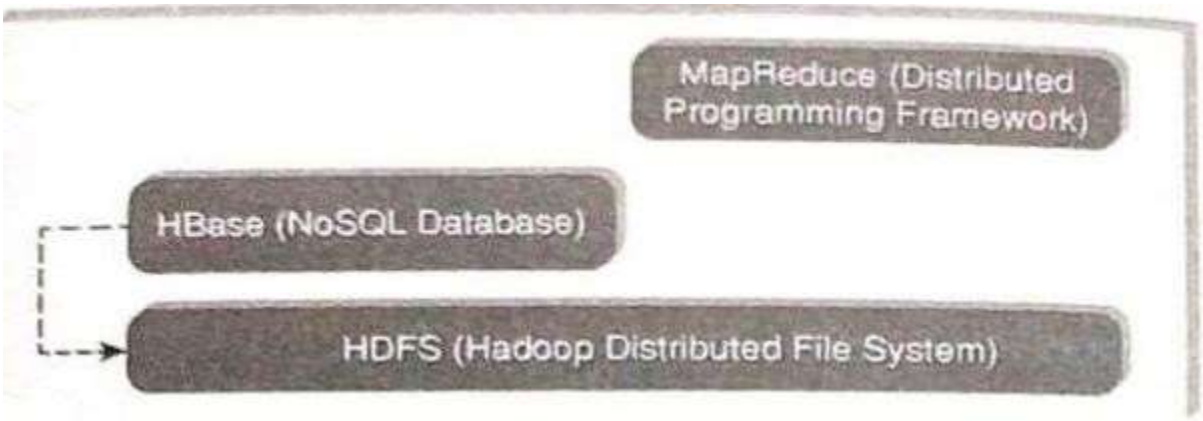


Figure 2.24 HBase in Hadoop ecosystem

VIDEO LINKS

Unit – II

R.M.K GROUP OF INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

10. VIDEO LINKS

Sl. No.	Topic	Video Link
1	Introducing Hadoop	https://www.youtube.com/watch?v=aReuLtY0YMI
2	RDBMS versus Hadoop	https://www.youtube.com/watch?v=fZD_sn1Sj8U
3	Hadoop Overview	https://www.youtube.com/watch?v=iANBytZ26MI
4	HDFS (Hadoop Distributed File System)	https://www.youtube.com/watch?v=GJYEsEEfjvk
5	Processing Data with Hadoop	https://www.youtube.com/watch?v=GRP-cGbJSCs
6	Managing Resources and Applications with Hadoop YARN	https://www.youtube.com/watch?v=KqaPMCMHH4_g
7	Interacting with Hadoop Ecosystem	https://www.youtube.com/watch?v=p0TdBqIt3fg

GROUP OF INSTITUTIONS

11. ASSIGNMENT : UNIT – II

1. Analyze why Hadoop moves **computation to data** rather than data to computation. If a TaskTracker fails to find a local block copy, evaluate the impact on **Network Bandwidth** and explain how **Rack Awareness** mitigates this performance dip.(CO2,K4)
2. HDFS is designed for "streaming" large files. Evaluate the architectural strain on the **NameNode's Metadata Heap** when storing millions of 10KB files versus a few 100GB files. Propose a solution using **Hadoop Archives (HAR)** or **SequenceFiles**. (CO2,K5)
3. Contrast how Hadoop and traditional RDBMS handle node failures. Specifically, analyze the role of **Speculative Execution** in managing "straggler" tasks and determine its impact on overall job completion time in a congested cluster. (CO2,K4)
4. Critically evaluate the **YARN Capacity Scheduler** in a multi-tenant environment. If a high-priority job requires immediate resources held by a low-priority job, argue the merits of **Container Preemption** versus waiting for task completion.(CO2,K5)
5. A 2.5GB dataset is stored in HDFS (128MB blocks, Replication 3).(CO2,K5)

- **Calculate:** The total number of blocks and the total physical space consumed.
- **Evaluate:** Explain the sequence of **FileSystem API** calls during a read operation and justify why increasing the block size might improve throughput for this specific dataset.

PART-A Q&A
UNIT-II

R.M.K GROUP OF INSTITUTIONS

12. PART A : Q & A : UNIT – II

1. What is Hadoop ? (CO2 , K1)
   i. Hadoop is an open source software programming framework for storing a large amount of data and performing the computation.
   ii. Its framework is based on Java programming with some native code in C and shell scripts.

1. List the key aspects of hadoop. (CO2 , K1)

The following are the key aspects of Hadoop:

- ➢ Open source software
- ➢ Framework
- ➢ Distributed
- ➢ Massive storage
- ➢ Faster processing

3. What are the core components of hadoop? (CO2 , K1)

Hadoop Core Components

1. HDFS:

(a) Storage component.

(b) Distributes data across several nodes.

(c) Natively redundant.

2. MapReduce:

(a) Computational framework.

(b) Splits a task across multiple nodes.

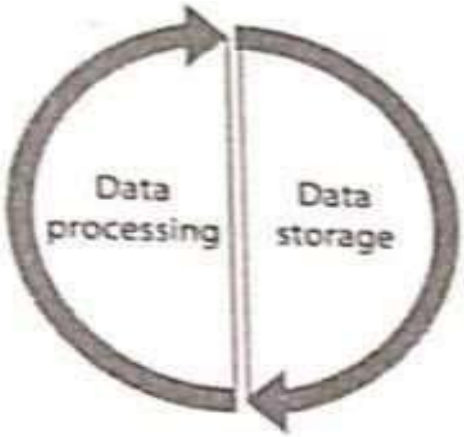
(c) Processes data in parallel.

R.M.K GROUP OF INSTITUTIONS

12. PART A : Q & A : UNIT – II

4. What is Hadoop Conceptual Layer? (CO2 , K1)

It is conceptually divided into **Data Storage Layer** which stores huge volumes of data and **Data Processing Layer** which processes data in parallel to extract richer and meaningful insights from data.



5. What is pig in Hadoop ecosystem?
   - Pig is a high-level platform in the Hadoop ecosystem designed for processing and analyzing large datasets. It uses a scripting language called Pig Latin, which is a simpler alternative to writing complex MapReduce programs.
   - Pig abstracts the complexity of MapReduce, making it easier to work with large-scale data processing tasks. It is commonly used for ETL (Extract, Transform, Load) operations and data analysis.

6. How sqoop is different from hbase? (CO2 , K1)
   - Sqoop is a tool which helps to transfer data between Hadoop and Relational Databases.
   - With the help of Sqoop, we can import data from RDBMS to HDFS and vice-versa.
   - HBase is a NoSQL database for Hadoop.
   - HBase is column-oriented NoSQL database. It is used to store **billions of rows and millions of columns**.

R.M.K GROUP OF INSTITUTIONS

12. PART A : Q & A : UNIT – II

7. What is the use of hive in hadoop ecosystem? (CO2 , K1)

- Hive is a Data Warehousing Layer on top of Hadoop.
- Analysis and queries can be done using an SQL-like language.
- Hive can be used to do ad-hoc queries, summarization, and data analysis.

8. List the Daemons that are part of YARN Architecture. (CO2 , K1)

(a) Global ResourceManager

- Scheduler
- ApplicationManager

(b) NodeManager

(c) Per-application ApplicationMaster

9. What are the major components of HDFS 2 ? (CO2 , K1)

- HDFS 2 consists of two major components

  (a) namespace

  (b) blocks storage service.
- Namespace service takes care of file-related operations, such as creating files, modifying files, and directories. The block storage service handles data node cluster management, replication.

10. What are the features of HDFS 2 ? (CO2 , K1)

- HDFS 2 Features

  (a) Horizontal scalability.

  (b) High availability

11. What is Active and passive NameNode ? (CO2 , K1)

- High availability of NameNode is obtained with the help of **Passive Standby NameNode.**

12. PART A : Q & A : UNIT – II

- In Hadoop 2.x, Active-passive NameNode handles failover
- Passive NameNode reads edits from shared storage and keeps updated metadata information.
- In case of Active NameNode failure, Passive NameNode becomes an Active NameNode automatically. Then it starts writing to the shared storage.

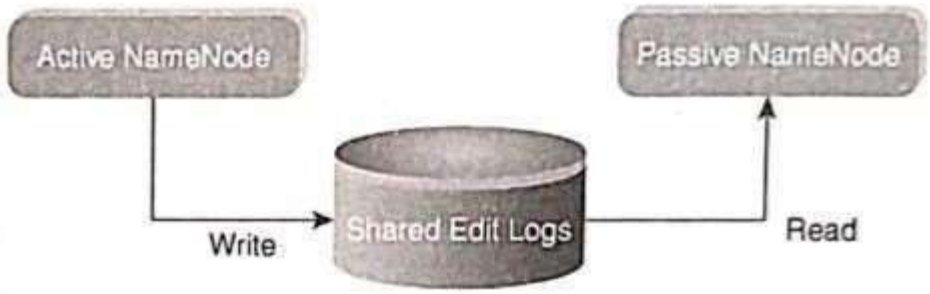


Figure 5.26 Active and Passive NameNode interaction.

12. What are the Limitations of Hadoop 1.0 Architecture? (CO2 , K1

In Hadoop 1.0, HDFS and MapReduce are Core Components, while other components are built around the core.

1. Single NameNode is responsible for managing entire namespace for Hadoop Cluster.

2. It has a restricted processing model which is suitable for batch-oriented MapReduce jobs.

3. Hadoop MapReduce is not suitable for interactive analysis.

4. Hadoop 1.0 is not suitable for machine learning algorithms, graphs, and other memory intensive algorithms.

12. PART A : Q & A : UNIT – II

13.What is HDFS Limitation ? What do you understand by HDFS Federation.
(CO2, K1)

- NameNode saves all its file metadata in main memory. Although the main memory today is not as small and as expensive as it used to be two decades ago, still there is a limit on the number of objects that one can have in the memory on a single NameNode.
- The NameNode can quickly become overwhelmed with load on the system increasing.
- In Hadoop 2.x, this is resolved with the help of **HDFS Federation**.

14. What is the difference between SQL and MapReduce. (CO2, K1)

- SQL is a query language used to manage and retrieve data from relational databases, while MapReduce is a programming model used to process large datasets in a distributed environment.

- SQL is best for structured data with predefined schemas, whereas MapReduce handles large-scale unstructured or semi-structured data across clusters.

15. What are the three important classes of MapReduce? (CO2, K1)

➢ MapReduce Programming requires three things.

1. Driver Class: This class specifies Job Configuration details.
2. Mapper Class: This class overrides the Map Function based on the problem statement.
3. Reducer Class: This class overrides the Reduce Function based on the problem statement.

R.M.K GROUP OF INSTITUTIONS

12. PART A : Q & A : UNIT – II

16. List the Daemons that are part of YARN Architecture.(CO2, K2)

The daemons that are part of the YARN (Yet Another Resource Negotiator) Architecture are:

- ResourceManager (RM) – Manages the resources in the cluster and allocates resources to applications.
- NodeManager (NM) – Runs on each node, monitors resource usage, and reports to the ResourceManager.
- ApplicationMaster (AM) – Manages the lifecycle of a specific application and negotiates resources from the ResourceManager.
- Container – A resource allocation unit that runs on a NodeManager to execute the tasks of an application.

17. Which daemon is responsible for executing overall MapReduce job? (CO2, K1)

- JobTracker provides connectivity between Hadoop and our application. When we submit code to cluster, JobTracker creates the execution plan by deciding which task to assign to which node. It also monitors all the running tasks.
- JobTracker is a master daemon responsible for executing overall MapReduce job.

18. What is MapReduce ? (CO2, K1)

i. MapReduce divides a data analysis task into two parts - **map** and **reduce**.

ii. For example, if there are two mappers and one reducer. Each mapper works

on the partial dataset that is stored on that node and the reducer combines the output from the mappers to produce the reduced result set

R.M.K GROUP OF INSTITUTIONS

12. PART A : Q & A : UNIT – II

19. What are the Special Features of HDFS ? (CO2, K1)

1. Data Replication:

There is absolutely no need for a client application to track all blocks. It directs the client to the nearest replica to ensure high performance.

2. Data Pipeline:

A client application writes a block to the first DataNode in the pipeline. Then this DataNode takes over and forwards the data to the next node in the pipeline. This process continues for all the data blocks, and subsequently all the replicas are written to the disk.

20. What is TaskTracker? (CO2, K1)

TaskTracker is a daemon in the MapReduce framework that runs on each node in a Hadoop cluster. It is responsible for executing individual Map and Reduce tasks, reporting the status of those tasks to the JobTracker, and managing resource usage for task execution on its node.

21. Why do we need Hadoop for Big Data Analytics? (CO2, K3)

Hadoop is needed for Big Data Analytics because it provides a scalable, distributed environment for storing and processing vast amounts of data.

Key reasons include:

- ➢ Fault Tolerance – Automatically replicates data to ensure reliability in case of hardware failures.
- ➢ Cost Efficiency – Uses commodity hardware to store and process large datasets, making it affordable for handling Big Data.

13. PART B QUESTIONS : UNIT – II

1. Describe the difference between RDBMS and Hadoop while processing the data. (CO2, K1)
2. How does Hadoop work? Describe the overview of hadoop. (CO2, K1)
3. Write short notes on Interacting with Hadoop Ecosystem. (CO2, K1)
4. Explain YARN Architecture in detail. (CO2, K1)
5. Explain the significances of Hadoop distributed file systems (HDFS) and its application (CO2,K1)
6. Explain Managing Resources and Applications with Hadoop YARN. (CO2, K1)
7. Write the Word Count MapReduce Programming using Java. (CO2, K1)
8. How Does MapReduce Work? (CO2, K1)
9. Write short notes on Working with HDFS Commands. (CO2, K1)
10. Explain the different features of RDBMS with example. (CO2,K2)

R.M.K GROUP OF INSTITUTIONS

14. SUPPORTIVE ONLINE CERTIFICATION COURSES

NPTEL : https://nptel.ac.in/courses/106104189

coursera : https://www.coursera.org/learn/hadoop

Udemy : https://www.udemy.com/course/the-ultimate-hands-on-hadoop-tame-your-big-data/

Mooc : https://www.mooc-list.com/tags/hadoop

edx : https://www.edx.org/learn/hadoop

R.M.K GROUP OF INSTITUTIONS

15. REAL TIME APPLICATIONS

1. Finance sectors

Financial organizations use hadoop for fraud detection and prevention. They use Apache Hadoop for reducing risk, identifying rogue traders, analyzing fraud patterns. Hadoop helps them to precisely target their marketing campaigns on the basis of customer segmentation.

2. Security and Law Enforcement

The USA national security agency uses Hadoop in order to prevent terrorist attacks and to detect and prevent cyber-attacks. Big Data tools are used by the Police forces for catching criminals and even predicting criminal activity. Hadoop is used by different public sector fields such as defense, intelligence, research, cybersecurity, etc.

3. Companies use Hadoop for understanding customers requirements

The most important application of Hadoop is understanding Customer' requirements. Different companies such as finance, telecom use Hadoop for finding out the customer's requirement by examining a big amount of data and discovering useful information from these vast amounts of data. By understanding customers behaviors, organizations can improve their sales.

4. Hadoop Applications in Retail industry

Retailers both online and offline use Hadoop for improving their sales. Many e-commerce companies use Hadoop for keeping track of the products bought together by the customers. On the basis of this, they provide suggestions to the customer to buy the other product when the customer is trying to buy one of the relevant products from that group.

For example, when a customer tries to buy a mobile phone, then it suggests a customer for the mobile back cover, screen guard.

Also, Hadoop helps retailers to customize their stocks based on the predictions that came from different sources such as Google search, social media websites, etc. Based on these predictions retailers can make the best decision which helps them to improve their business and maximize their profits.

5. Real-time analysis of customers data

Hadoop can analyze customer data in real-time. It can track clickstream data as it's for storing and processing high volumes of clickstream data. When a visitor visits a website, then Hadoop can capture information like from where the visitor originated before reaching a particular website, the search used for landing on the website.

Hadoop can also grab data about the other webpages in which the visitor shows interest, time spent by the visitor on each page, etc. This is the analysis of website performance and user engagement.

R.M.K GROUP OF INSTITUTIONS

16. CONTENTS BEYOND SYLLABUS : UNIT – II

Apache Spark

Apache Spark is an open-source, distributed processing system used for big data workloads. It utilizes in-memory caching, and optimized query execution for fast analytic queries against data of any size. It provides development APIs in Java, Scala, Python and R, and supports code reuse across multiple workloads—batch processing, interactive queries, real-time analytics, machine learning, and graph processing.

Spark Core is the base of the whole project. It provides distributed task dispatching, scheduling, and basic I/O functionalities. Spark uses a specialized fundamental data structure known as RDD (Resilient Distributed Datasets) that is a logical collection of data partitioned across machines.

RDDs can be created in two ways; one is by referencing datasets in external storage systems and second is by applying transformations (e.g. map, filter, reducer, join) on existing RDDs.

The RDD abstraction is exposed through a language-integrated API. This simplifies programming complexity because the way applications manipulate RDDs is similar to manipulating local collections of data.

Spark Shell

Spark provides an interactive shell – a powerful tool to analyze data interactively. It is available in either Scala or Python language. Spark's primary abstraction is a distributed collection of items called a Resilient Distributed Dataset (RDD).

R.M.K GROUP OF INSTITUTIONS

RDDs can be created from Hadoop Input Formats (such as HDFS files) or by transforming other RDDs.

Features of Apache Spark

Apache Spark has following features.

- Speed – Spark helps to run an application in Hadoop cluster, up to 100 times faster in memory, and 10 times faster when running on disk. This is possible by reducing number of read/write operations to disk. It stores the intermediate processing data in memory.
- Supports multiple languages – Spark provides built-in APIs in Java, Scala, or Python. Therefore, we can write applications in different languages. Spark comes up with 80 high-level operators for interactive querying.
- Advanced Analytics – Spark not only supports 'Map' and 'reduce'. It also supports SQL queries, Streaming data, Machine learning (ML), and Graph algorithms.

R.M.K GROUP OF INSTITUTIONS

Assessment Schedule (Proposed Date & Actual Date)

R.M.K GROUP OF INSTITUTIONS

17.Assessment Schedule (Proposed Date & Actual Date)

Sl. No.	ASSESSMENT	Proposed Date	Actual Date
1	FIRST INTERNAL ASSESSMENT		
2	SECOND INTERNAL ASSESSMENT		
3	MODEL EXAMINATION		
4	END SEMESTER EXAMINATION		

R.M.K GROUP OF INSTITUTIONS

18. PRESCRIBED TEXT BOOKS & REFERENCE BOOKS

TEXT BOOKS:

1. Subhashini Chellappan, Seema Acharya, "Big Data and Analytics", 2nd edition, Wiley Publications, 2019.
2. Suresh Kumar Mukhiya and Usman Ahmed, "Hands-on Exploratory Data Analysis with Python", Packt publishing, March 2020.
3. Jure Leskovek, Anand Rajaraman and Jefrey Ullman," Mining of Massive Datasets. v2.1", Cambridge University Press,2019.
4. Glenn J. Myatt, Wayne P. Johnson, Making Sense of Data II : A Practical Guide To Data Visualization, Advanced Data Mining Methods, and Applications, Wiley 2009.

REFERENCES:

1. Nelli, F., Python Data Analytics: with Pandas, NumPy and Matplotlib, Apress, 2018.
2. Bart Baesens," Analytics in a Big Data World: The Essential Guide to Data Science and its Applications", John Wiley & Sons, 2014
3. Min Chen, Shiwen Mao, Yin Zhang, Victor CM Leung, Big Data: Related Technologies, Challenges and Future Prospects, Springer, 2014.
4. Michael Minelli, Michele Chambers, Ambiga Dhiraj, "Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends", John Wiley & Sons, 2013.
5. Marcello Trovati, Richard Hill, Ashiq Anjum, Shao Ying Zhu, "Big Data Analytics and cloud computing – Theory, Algorithms and Applications", Springer International Publishing, 2016

R.M.K GROUP OF INSTITUTIONS

19. **MINI PROJECT SUGGESTION**

1. Analyze patient admission logs to correlate "Wait Times" with "Doctor-to-Patient Ratios" across different hospital wings.(CO2,K4)
2. Identify suspicious transaction patterns (e.g., velocity checks or geographic "impossible travel").(CO2,K5)
3. Process clickstream data to identify at what stage (Add-to-cart vs. Checkout) users drop off. (CO2,K4)
4. Aggregate smart-meter data from thousands of households to identify power grid inefficiencies.(CO2,K5)
5. Analyze global news feeds and social mentions to predict potential disruptions in supply chain routes (CO2,K4).

R.M.K GROUP OF INSTITUTIONS

19. MINI PROJECT SUGGESTION

Mini Project:

The project should contain the following components

- Realtime dataset
- Data preparation & Transformation
- Handling missing Data
- Data Storage
- Algorithm for data analytics
- Data visualization: Charts, Heatmap, Crosstab, Treemap

Some of the mini projects are:

1. Log File Analyzer

- **Description**: Parse server log files to extract insights like most visited pages, error counts, user IP frequency.
- **Skills**: String parsing, log analysis, MapReduce.
- **Dataset**: Apache access logs or simulated logs.

---

2. Retail Transaction Analysis

- **Description**: Analyze sales data to find total sales per product, top-selling items, or peak purchase times.
- **Skills**: Aggregation, grouping, MapReduce joins.
- **Dataset**: Kaggle's Retail datasets.

---

3. Twitter Hashtag Counter

- **Description**: Count the frequency of hashtags used in a tweet dataset.
- **Skills**: Text extraction, filtering, frequency analysis.
- **Dataset**: Twitter Streaming API or sample datasets.

---

4. Movie Ratings Aggregator

- **Description**: Use a dataset like MovieLens to compute average ratings, highest-rated movies, etc.
- **Skills**: MapReduce, data joins, aggregation.
- **Dataset**: MovieLens Dataset

R.M.K GROUP OF INSTITUTIONS

20. GATE QUESTIONS

1. HDFS Architecture & NameNode

Question: In a Hadoop cluster, which of the following statements is **FALSE** regarding the NameNode?

A) It stores the metadata of the files in the HDFS.

B) It stores the actual data blocks of the files.

C) It manages the file system namespace.

D) It keeps track of the health of DataNodes via heartbeats.

Answer: B

Solution: The NameNode only manages metadata (file names, permissions, and the mapping of file blocks to DataNodes). The actual data blocks are stored on the DataNodes.

2. Block Replication & Storage

Question: If the default block size in HDFS is 128 MB and a file of size 150 MB is uploaded with a replication factor of 3, how many total blocks will be stored in the cluster?

A) 2

B) 3

C) 6

D) 4

Answer: C

Solution: - Number of blocks for 150 MB: [150/128]= 2 blocks (one 128MB block and one 22MB block).

•Total blocks with replication: 2 blocks X 3 replicas = 6 blocks

3. MapReduce Execution Flow

Question: In a MapReduce job, which phase is responsible for ensuring that all values associated with the same key are sent to the same Reducer?

A) Mapping Phase

B) Combining Phase

C) Shuffle and Sort Phase

D) Partitioning Phase

Answer: D

Solution: The Partitioner determines which reducer a specific (key, value) pair is sent to. The Shuffle and Sort phase then physically moves and organizes this data for the reducer.

R.M.K GROUP OF INSTITUTIONS

4. HDFS Fault Tolerance

Question: When a DataNode fails, how does the NameNode handle the loss of data blocks?

A) It immediately shuts down the entire cluster to prevent data corruption.

B) It identifies the under-replicated blocks and commands other DataNodes to create new replicas.

C) It restores the data from the Secondary NameNode's hard drive.

D) It asks the client to re-upload the missing data.

Answer: B

Solution: Hadoop is self-healing. The NameNode tracks replication factors; if a node goes down, it ensures the replication factor is maintained by copying blocks from healthy nodes to other active nodes.

5. YARN Architecture

Question: Which component in YARN (Yet Another Resource Negotiator) is responsible for negotiating resources from the ResourceManager and working with the NodeManager(s) to execute and monitor tasks?

A) ApplicationMaster

B) Scheduler

C) Containers

D) ResourceTracker

Answer: A

Solution: The ApplicationMaster is a per-application structure that negotiates resources from the ResourceManager and works with the NodeManager to execute and monitor the tasks.

6. Combiner Functions

Question: What is the primary purpose of using a "Combiner" in a MapReduce job? A) To replace the Reducer entirely. B) To reduce the amount of data transferred across the network between Mapper and Reducer. C) To increase the number of input splits. D) To handle the sorting of keys in the Map phase.

Answer: B Solution: A Combiner acts as a "mini-reducer" that runs on the Mapper node to aggregate data locally, thereby reducing network I/O.

R.M.K GROUP OF INSTITUTIONS

7. Secondary NameNode

Question: Which of the following best describes the role of the Secondary NameNode?

A) It acts as a "Hot Standby" that takes over immediately if the Primary NameNode fails.

B) B) It performs periodic checkpoints by merging the EditLog with the FsImage.

C) C) It provides a backup for the DataNodes.) It is used to store high-priority data blocks.

Answer: B Solution: The Secondary NameNode is NOT a high-availability backup. Its main job is to merge the FsImage and EditLogs to prevent the EditLog from becoming too large, which speeds up NameNode restart times.

8. MapReduce Logic (InputSplits)

Question: How is the number of Map tasks usually determined in a Hadoop job?

A) By the number of Reducers specified by the user.

B) By the number of InputSplits calculated by the InputFormat.

C) By the number of DataNodes in the cluster.

D) It is always a fixed number (100).

Answer: B Solution: Each InputSplit is processed by one Map task. The number of splits usually corresponds to the number of blocks the input data occupies in HDFS.

9. HDFS Read Operation

Question: During an HDFS file read operation, what information does the NameNode provide to the client?

A) The actual content of the file.

B) A single DataNode address where the entire file is stored

R.M.K GROUP OF INSTITUTIONS

C) The addresses of DataNodes that host the blocks of that file, sorted by proximity to the client.

D) A decryption key to access the DataNodes.

Answer: C Solution: The NameNode provides the block locations (DataNode addresses). To optimize performance, it sorts these addresses based on network topology (Rack Awareness) so the client reads from the nearest node.

10. Speculative Execution

Question: "Speculative Execution" in Hadoop refers to:

A) Executing tasks that might be needed in the future based on predictive analytics.

B) Running duplicate copies of slow-moving tasks on different nodes to reduce overall job time.

C) Encrypting data before it is written to the HDFS.

D) Testing MapReduce code on a small sample before running it on the full dataset.

Answer: B Solution: If a task is running significantly slower than the average (a "straggler"), Hadoop "speculatively" launches another instance of the same task on a different node. Whichever finishes first is accepted, and the other is killed.

R.M.K GROUP OF INSTITUTIONS

21. AI based Application

1. Intelligent Data Storage and Management

The Hadoop Distributed File System (HDFS) stores massive datasets across multiple nodes. AI techniques are applied to optimize storage, retrieval, and data organization.

Applications:

Smart data placement using machine learning
Data redundancy optimization
Predicting node failures
Intelligent data retrieval

2.I**ntelligent System Maintenance**

AI monitors Hadoop clusters to detect anomalies and failures.

Applications:

Predictive maintenance of nodes
Log analysis using AI
Failure prediction
Performance tuning

3. Smart Data Protection

AI enhances security in Hadoop by analyzing access patterns and detecting threats.

Applications:

Intrusion detection
User behavior analytics
Data access anomaly detection
Fraud prevention

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.